

Práctica Data Warehouse & SQL

A lo largo de esta práctica pondremos en práctica los conocimientos adquiridos en el módulo **Data Warehouse & SQL Advanced**.

Encontrarás una serie de enunciados. Cada uno te pedirá escribir **una o varias queries**

SQL que te permita responder a la pregunta planteada. Para cada enunciado:

- entrega la query/s como respuesta principal
- si quieres, añade una breve interpretación del resultado (extra, para practicar análisis de datos)

Para esta práctica trabajaremos con un fichero con información de vuelos: flights.csv. Súbelo a tu base de datos en PostgreSQL... ¡y empezamos!

Enunciado 1.

Explora el fichero flights y analiza:

1. Cuántos registros hay en total.

```
select
```

```
    count(flight_row_id) as total
```

```
from flights;
```

– hay 1209

2. Cuántos vuelos distintos hay.

```
select
```

```
    count(distinct unique_identifier) as flights_distinct
```

```
from flights;
```

–hay 266

3. Cuántos vuelos tienen más de un registro.

```
with base as (
```

```
    select
```

```
        unique_identifier,
```

```
        count(flight_row_id) as unique_identifier_total
    from flights
    group by unique_identifier
    having count(flight_row_id) > 1
)
```

```
select
    count(unique_identifier) as total_flights
from base;
```

–250

Enunciado 2.

Por qué hay registros duplicados para un mismo vuelo. Para ello, selecciona varios vuelos y analiza la evolución temporal de cada vuelo.

1. Qué información cambia de un registro a otro.

Estos son los buenos que he cogido para hacer este enunciado:

IB-180-20240711-MAD-BCN

VY-193-20241018-BCN-AMS

AZ-7760-20220611-FCO-LIS

AV-47-20211101-MAD-BOG

```
select
    flight_row_id,
    unique_identifier,
    local_departure,
    local_actual_arrival,
    local_arrival,
    local_actual_arrival,
    gmt_departure,
    gmt_actual_departure,
```

```
gmt_arrival,  
gmt_actual_arrival,  
delay_mins,  
coalesce(arrival_status,'Sin estado') as arrival_status,  
created_at,  
updated_at  
from flights  
where unique_identifier = 'AZ-7760-20220611-FCO-LIS'  
order by flight_row_id asc;
```

– No todos los vuelos tienen created_at y updated_at.

Al no tener todos los vuelos created_at y updated_at voy a tomar como referencia para ordenar “flight_row_id”.

Para todos los vuelos duplicados con n registros siempre se cumple el mismo patrón.

El primer registro que se añade tiene los campos local_actual_departure, local_actual_arrival, gmt_actual_departure, gmt_actual_arrival, delay_mins en NULL.

Sin tener mucha idea de como funciona un aeropuerto. Me puedo hacer la idea de que esto pasa cuando se prepara un vuelo.

El otro patrón que se cumple es el último registro que ya tiene todos esos datos rellenos. Que puede indicar que el vuelo ya ha llegado a su destino.

Para los vuelos con más de dos registros los registros que están entre el primero y el ultimo varia dependiendo el vuelo.

Enunciado 3.

Evalúa la calidad del dato. La calidad del dato nos indica si la información es consistente, completa, coherente y representa una realidad verosímil. Para ello debemos establecer unos criterios:

1. La información de created_at debe ser única para cada vuelo aunque tenga más de un registro.

```
with base as (  
    select  
        unique_identifier,  
        min(created_at) as fecha_min
```

```

    from flights
    where created_at is not null
    and updated_at is not null
    group by unique_identifier
)
select
    count(fecha_min) as total_fechas_created,
    count(distinct fecha_min) as total_fechas_distintas_created
from base;

```

He quitado los registros null que ya en el ejercicio anterior comente que habia registros null en created_at.

En los registros duplicados me quedo con la primera fecha. Luego comparó el nuevo número de registros con el número de registros distintos y veo que no da el mismo resultado. Luego no hay coherencia.

2. La información de updated_at deber ser igual o más que la información de created_at, lo que nos indica coherencia y consistencia

```

select
    unique_identifier,
    min(created_at) as fecha_min,
    updated_at
from flights
where created_at is not null
and updated_at is not null
group by unique_identifier, updated_at
having updated_at < min(created_at);

```

Devuelve 0 resultados por lo que si seria consistente y coherente. Pero quitando los valores nulos

Enunciado 4

El último estado de cada vuelo. Cada vuelo puede aparecer varias veces en el dataset, para avanzar con nuestro análisis necesitamos quedarnos solo con el último registro de cada vuelo.

Puedes crear una tabla o vista resultante de esta query en tu base de datos local, la utilizaremos en los siguientes enunciados. Si prefieres no guardar la última información, tendrás que hacer uso de esa query como una CTE en los enunciados siguientes.

```
with base as (  
    select  
        unique_identifier,  
        flight_row_id,  
        row_number() over(partition by unique_identifier order by flight_row_id desc)  
as rn  
    from flights  
)  
ventana_ultimos_registros as (  
    select  
        unique_identifier,  
        flight_row_id  
    from base  
    where rn = 1  
)  
  
select *  
from ventana_ultimos_registros
```

al tener campos null created_at y updated_at he decidido utilizar para row_number el campo flight_row_id

Enunciado 5.

Considerando que los campos local_departure y local_actual_departure son necesarios para el análisis, valida y reconstruye estos valores siguiendo estas reglas:

1. Si local_departure es nulo, utiliza created_at.

2. Si local_actual_departure es nulo, utiliza local_departure. Si este también es nulo, utiliza created_at.

Crea dos nuevos campos:

- effective_local_departure
- effective_local_actual_departure

Extra: Realiza las validaciones para los campos local_arrival y local_actual_arrival.

```
with base as (  
    select  
        unique_identifier,  
        flight_row_id,  
        delay_mins,  
        row_number() over(partition by unique_identifier order by flight_row_id desc)  
as rn  
    from flights  
)  
  
ventana_ultimos_registros as (  
    select  
        unique_identifier,  
        flight_row_id,  
        delay_mins  
    from base  
    where rn = 1  
)  
  
select  
    ven.unique_identifier,  
    case  
        when fli.local_departure is null then fli.created_at  
        else fli.local_departure  
    end as effective_local_departure,  
    case
```

```

        when fli.local_actual_departure is null and fli.local_departure is null then
fli.created_at
        when fli.local_actual_departure is null then fli.local_departure
        else fli.local_actual_departure
        end as effective_local_actual_departure,

    case

        when fli.local_arrival is null then fli.created_at
        else fli.local_arrival
        end as effective_local_arrival,

    case

        when fli.local_actual_arrival is null and fli.local_arrival is null then
fli.created_at
        when fli.local_actual_arrival is null then fli.local_arrival
        else fli.local_actual_arrival
        end as effective_local_actual_arrival

from

    ventana_ultimos_registros ven

left join flights fli
on ven.flight_row_id = fli.flight_row_id

```

Enunciado 6.

Análisis del estado del vuelo. Haciendo uso del resultado del enunciado 4, analiza los estados de los vuelos.

1. Qué estados de vuelo existen

```

with base as (

    select

        unique_identifier,

        flight_row_id,

        delay_mins,

```

```

        row_number() over(partition by unique_identifier order by flight_row_id desc)
as rn
        from flights
    ),
    ventana_ultimos_registros as (
        select
            unique_identifier,
            flight_row_id,
            delay_mins
        from base
        where rn = 1
    )
select
    count(distinct fli.arrival_status) as numero_estados
from
    ventana_ultimos_registros ven
left join flights fli
on ven.flight_row_id = fli.flight_row_id
where fli.arrival_status is not null;

```

– 5 estados

2. Cuántos vuelos hay por cada estado

```

with base as (
    select
        unique_identifier,
        flight_row_id,
        delay_mins,
        row_number() over(partition by unique_identifier order by flight_row_id desc)
as rn

```



```

        from flights
    ),
    ventana_ultimos_registros as (
        select
            unique_identifier,
            flight_row_id,
            delay_mins
        from base
        where rn = 1
    )
    select
        fli.arrival_status as estados,
        count(ven.unique_identifier) as numero_vuelos

    from
        ventana_ultimos_registros ven
    left join flights fli
    on ven.flight_row_id = fli.flight_row_id
    where fli.arrival_status is not null
    group by fli.arrival_status;

```

– y 11 null que los he quitado

¿Podrías decir qué significa las siglas de cada estado?

CX. Tripulación (Crew). Problemas con disponibilidad, descanso legal, cambios de turno, etc.

DY. Razones comerciales / operativas de la aerolínea. Ajustes de programación, conexión de pasajeros, reacomodo de equipo, etc.

EY. Instalaciones o servicios del aeropuerto. Puertas, manejo en tierra, equipaje, servicios aeroportuarios.

NS. Seguridad / normativa nacional. Controles de seguridad, restricciones oficiales,

procedimientos regulatorios.

OT. Otros (Other). Cualquier causa que no encaja en categorías estándar.

Enunciado 7.

País de salida de cada vuelo. Tienes disponible un csv. con información de aeropuertos airports.csv. Haciendo uso del resultado del enunciado 4, analiza los aeropuertos de salida.

1. De qué país despegan los vuelos

```
with base as (  
    select  
        unique_identifier,  
        flight_row_id,  
        delay_mins,  
        row_number() over(partition by unique_identifier order by flight_row_id desc)  
as rn  
    from flights  
,  
ventana_ultimos_registros as (  
    select  
        unique_identifier,  
        flight_row_id,  
        delay_mins  
    from base  
    where rn = 1  
)  
select  
    ven.unique_identifier as vuelos,  
    air.country as pais  
from  
    ventana_ultimos_registros ven  
left join flights fli
```

```
on ven.flight_row_id = fli.flight_row_id
inner join airports air
on fli.departure_airport = air.airport_code;
```

– Hay aeropuertos que no están en la tabla airports. Por eso he utilizado un inner ya que entiendo que de los aeropuertos que no están en la tabla no son importantes

2. Cuántos vuelos despegan por país

```
with base as (
    select
        unique_identifier,
        flight_row_id,
        delay_mins,
        row_number() over(partition by unique_identifier order by flight_row_id desc)
as rn
    from flights
),
ventana_ultimos_registros as (
    select
        unique_identifier,
        flight_row_id,
        delay_mins
    from base
    where rn = 1
),
info_paises as (
    select
        ven.unique_identifier as vuelos,
        air.country as pais
```

```

        from
            ventana_ultimos_registros ven
        left join flights fli
        on ven.flight_row_id = fli.flight_row_id
        inner join airports air
        on fli.departure_airport = air.airport_code
    )

select
    pais,
    count(vuelos) as numero_vuelos_por_pais
from info_paises
group by pais;

```

Enunciado 8.

Delay medio y estado de vuelo por país de salida. Haciendo uso del resultado del enunciado 4, analiza el estado y el delay/retraso medio con el objetivo de identificar si existen países que pueden presentar problemas operativos en los aeropuertos de salida.

1. Cuál es el delay medio por país

```

with base as (
    select
        unique_identifier,
        flight_row_id,
        delay_mins,
        row_number() over(partition by unique_identifier order by flight_row_id desc)
    as rn
    from flights
),

```

```

ventana_ultimos_registros as (
    select
        unique_identifier,
        flight_row_id,
        delay_mins
    from base
    where rn = 1
),
info_delay as (
    select
        coalesce(ven.delay_mins, 0) as delay_mins,
        air.country as pais

    from
        ventana_ultimos_registros ven
    left join flights fli
    on ven.flight_row_id = fli.flight_row_id
    inner join airports air
    on fli.arrival_airport = air.airport_code

)
select
    pais,
    round(avg(delay_mins),2) as delay_mins_media
from info_delay
group by pais;

```

– He cogido el aeropuerto de llegada (arrival_airport). He hecho un inner para quitarme los nulos de aeropuerto y los nulos de delay los he tomado como 0

2.Cuál es la distribución de estados de vuelos por país.

```
with base as (  
    select  
        unique_identifier,  
        flight_row_id,  
        delay_mins,  
        row_number() over(partition by unique_identifier order by flight_row_id desc)  
    as rn  
    from flights  
)  
ventana_ultimos_registros as (  
    select  
        unique_identifier,  
        flight_row_id,  
        delay_mins  
    from base  
    where rn = 1  
)  
info_estado as (  
    select  
        fli.arrival_status as estado,  
        air.country as pais  
  
    from  
        ventana_ultimos_registros ven  
    left join flights fli  
    on ven.flight_row_id = fli.flight_row_id
```

```
inner join airports air
on fli.arrival_airport = air.airport_code
where fli.arrival_status is not null

)

select
    pais,
    estado,
    count(estado) as numero_veces
from info_estado
group by pais, estado;
```

Extra:

Representa gráficamente la distribución de estados por país. Puedes dibujar un gráfico de barras o representarlo como creas que mejor se visualiza.

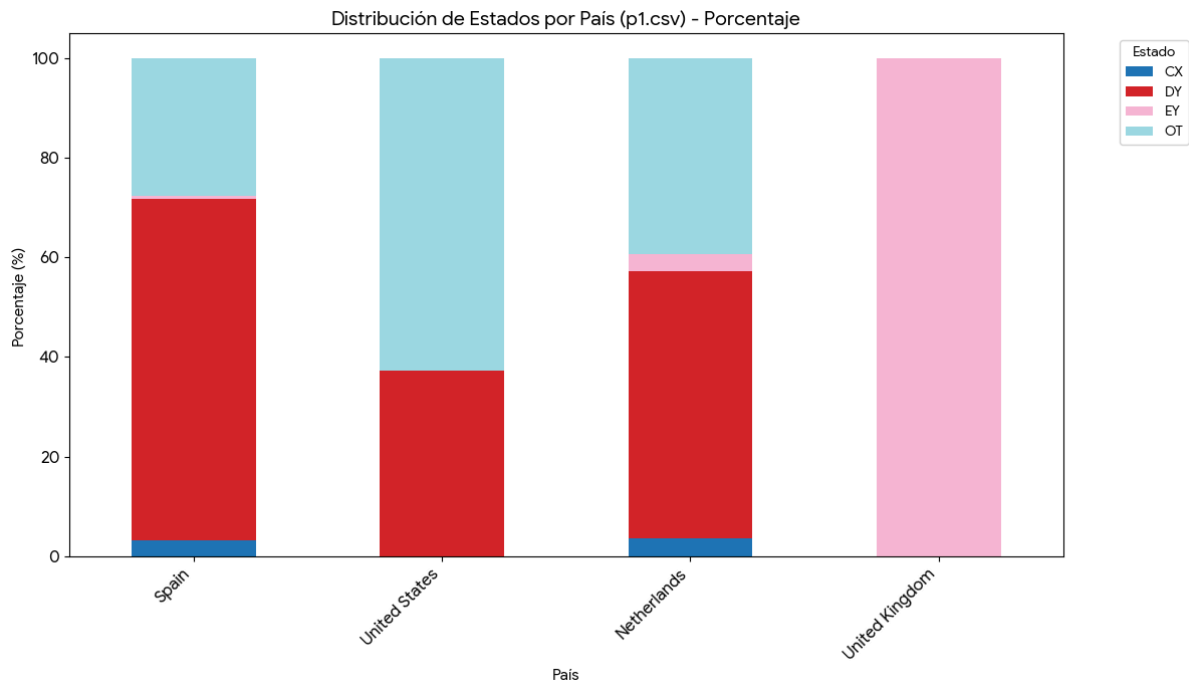


Diagrama por porcentaje de cada estado por país. Se ve que hay uno que solo tiene un estado

Enunciado 9.

El estado de vuelo por país y por época del año. Dado que no en todas las épocas del año las condiciones climatológicas son iguales, analiza si las estaciones del año impactan en el delay medio por país. Considera la siguiente clasificación de meses del año por época:

- **Invierno:** diciembre, enero, febrero
- **Primavera:** marzo, abril, mayo
- **Verano:** junio, julio, agosto
- **Otoño:** septiembre, octubre, noviembre

with base as (

select

unique_identifier,

flight_row_id,

delay_mins,

row_number() over(partition by unique_identifier order by flight_row_id desc)

as rn


```

        from flights
    ),
    ventana_ultimos_registros as (
        select
            unique_identifier,
            flight_row_id,
            delay_mins
        from base
        where rn = 1
    ),
    info_fecha as (
        select
            air.country as pais,
            coalesce(fli.local_actual_arrival, fli.local_arrival) as fecha,
            coalesce(ven.delay_mins,0) as delay_mins

        from
            ventana_ultimos_registros ven
        left join flights fli
        on ven.flight_row_id = fli.flight_row_id
        inner join airports air
        on fli.arrival_airport = air.airport_code
        where fli.arrival_status is not null
    ),
    info_delay as (
        select
            pais,
            extract(month from fecha)::int as mes,

```

```

        delay_mins
    from info_fecha

),
info_media_delay as (
    select
        pais,
        delay_mins,
        case
            when mes = 12 or mes between 1 and 2 then 'Invierno'
            when mes between 3 and 5 then 'Primavera'
            when mes between 6 and 8 then 'Verano'
            when mes between 9 and 11 then 'Otoño'
        end as epoca
    from info_delay

)

```

```

select
    pais,
    epoca,
    round(avg(delay_mins),2) as delay_mins_media
from info_media_delay
group by pais, epoca
order by pais desc;

```

–He aplicado los datos arrival. Cogiendo local_actual_arrival y si es null local_arrival.

Los delays nulos los he cambiado a 0. El delay en los países varía en cada época del año.

Enunciado 10.

Frecuencia de actualización de los vuelos. Volviendo al análisis de la calidad del dataset, explora con qué frecuencia se registran actualizaciones de cada vuelo y calcula la frecuencia media de actualización por aeropuerto de salida.

```
with base as (  
    select  
        unique_identifier,  
        departure_airport,  
        coalesce(updated_at, created_at) as updated_at  
    from flights  
    where updated_at is not null  
)  
info_lag as (  
    select  
        unique_identifier,  
        departure_airport,  
        updated_at,  
        lag(updated_at) over (partition by unique_identifier order by updated_at desc)  
    as after_update_at  
    from base  
)  
info_cuenta as (  
    select  
        departure_airport,  
        updated_at - after_update_at as intervalo  
    from info_lag  
    where after_update_at is not null  
)
```

```
select
    departure_airport as vuelos,
    avg(intervalo)
from info_cuenta
group by vuelos;
```

Enunciado 11.

Consistencia del dato. El campo `unique_identifier` identifica el vuelo y se construye con: aerolínea, número de vuelo, fecha y aeropuertos. Para cada vuelo (último snapshot), comprueba si la información del `unique_identifier` es consistente con las columnas del dataset.

1. Crea un flag `is_consistent`.

```
with base as (
    select
        unique_identifier,
        flight_row_id,
        row_number() over(partition by unique_identifier order by flight_row_id desc)
as rn
    from flights
),
ventana_ultimos_registros as (
    select
        unique_identifier,
        flight_row_id
    from base
    where rn = 1
),
info_general as (
    select
        flight_row_id as id,
```

```

        unique_identifier,
        split_part(unique_identifier, '-', 1) as airline_code,
        split_part(unique_identifier, '-', 3) as date,
        split_part(unique_identifier, '-', 4) as departure_airport,
        split_part(unique_identifier, '-', 5) as arrival_airport
    from ventana_ultimos_registros
),
info_comparar as (
    select
        flight_row_id as id,
        airline_code,
        replace(replace(date_trunc('day', local_departure)::text, ' 00:00:00', ''), '-', '') as
date,
        departure_airport,
        arrival_airport
    from flights
)
select
    ig.unique_identifier as unique_identifier,
    case
        when ig.airline_code = ic.airline_code
        and ig.date = ic.date
        and ig.departure_airport = ic.departure_airport
        and ig.arrival_airport = ic.arrival_airport
        then true
        else false
    end as is_consistent
from info_general as ig
inner join info_comparar as ic

```

on ig.id = ic.id;

– me falta comparar con el numero de vuelo pero ese campo no lo tenemos en ninguna tabla

2. Calcula cuántos vuelos no son consistentes.

```
with base as (  
    select  
        unique_identifier,  
        flight_row_id,  
        row_number() over(partition by unique_identifier order by flight_row_id desc)  
as rn  
    from flights  
)  
  
ventana_ultimos_registros as (  
    select  
        unique_identifier,  
        flight_row_id  
    from base  
    where rn = 1  
)  
  
info_general as (  
    select  
        flight_row_id as id,  
        unique_identifier,  
        split_part(unique_identifier, '-', 1) as airline_code,  
        split_part(unique_identifier, '-', 3) as date,  
        split_part(unique_identifier, '-', 4) as departure_airport,  
        split_part(unique_identifier, '-', 5) as arrival_airport  
    from ventana_ultimos_registros
```

```

),
info_comparar as (
    select
        flight_row_id as id,
        airline_code,
        replace(replace(date_trunc('day', local_departure)::text, ' 00:00:00', ''), '-', '') as
date,
        departure_airport,
        arrival_airport
    from flights
),
info_contar as (
    select
        ig.unique_identifier as unique_identifier,
        case
            when ig.airline_code = ic.airline_code
            and ig.date = ic.date
            and ig.departure_airport = ic.departure_airport
            and ig.arrival_airport = ic.arrival_airport
            then true
            else false
        end as is_consistent
    from info_general as ig
    inner join info_comparar as ic
    on ig.id = ic.id
)
select
    count(unique_identifier) as total_no_consistentes
from info_contar

```

```
where is_consistent = false;
```

– 15

3. Usando la tabla airlines, muestra el nombre de la aerolínea y cuántos vuelos no consistentes tiene.

```
with base as (  
    select  
        unique_identifier,  
        flight_row_id,  
        row_number() over(partition by unique_identifier order by flight_row_id desc)  
    as rn  
    from flights  
)  
  
ventana_ultimos_registros as (  
    select  
        unique_identifier,  
        flight_row_id  
    from base  
    where rn = 1  
)  
  
info_general as (  
    select  
        flight_row_id as id,  
        unique_identifier,  
        split_part(unique_identifier, '-', 1) as airline_code,  
        split_part(unique_identifier, '-', 3) as date,  
        split_part(unique_identifier, '-', 4) as departure_airport,  
        split_part(unique_identifier, '-', 5) as arrival_airport  
    from ventana_ultimos_registros  
)
```



```

info_comparar as (
    select
        flight_row_id as id,
        airline_code,
        replace(replace(date_trunc('day', local_departure)::text, ' 00:00:00', ''), '-', '') as
date,
        departure_airport,
        arrival_airport
    from flights
),
info_airline as (
    select
        ig.unique_identifier as unique_identifier,
        ig.airline_code as ig_airline_code,

        case
            when ig.airline_code = ic.airline_code
            and ig.date = ic.date
            and ig.departure_airport = ic.departure_airport
            and ig.arrival_airport = ic.arrival_airport
            then true
            else false
        end as is_consistent
    from info_general as ig
    inner join info_comparar as ic
    on ig.id = ic.id
)

```

```
select
    a.name as nombre_aerolinia,
    count(unique_identifier) as total_no_consistentes
from info_airline as ia
inner join airlines as a
on ia.ig_airline_code = a.airline_code
where is_consistent = false
group by nombre_aerolinia;
```

– Una de las causas de que el vuelo no sea consistente es que la aerolínea no coincida con la del vuelo como pasa aquí `select * from flights where unique_identifier = 'BA-7770-20240620-LHR-MAD'`

Por eso para esta tarea he cogido la aerolínea que viene en el `unique_identifier` como su aerolínea real. Con esa aerolínea he realizado la tarea.

